



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**DESIGN AND DEVELOPMENT OF A SMART WASTE
MANAGEMENT
SYSTEM WITH AUTOMATED TRASH SEGREGATION**

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements

for the Course of

CSA1110 — OBJECT ORIENTED ANALYSIS AND DESIGN

to the award of the degree of

BACHELOR OF ENGINEERING

IN

INFORMATION TECHNOLOGY

Submitted by

NAVEEN KUMAR VV (192321018)

Under the Guidance of

Dr. A. MANIMARAN

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Mini Project entitled “Design and Development of a Smart Waste Management System with Automated Trash Segregation” has been carried out Naveen Kumar VV (192321018) under the guidance of Dr. A. Manimaran and is submitted in partial fulfilment of the requirements for the course CSA1110 — Object Oriented Analysis and Design of the Information Technology programme at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. Ramamurthy

Department of Computer Science and Engineering

SIMATS Engineering, Chennai – 602105.

COURSE FACULTY

Dr. A. Manimaran

Department of Computer Science and Engineering

SIMATS Engineering, Chennai – 602105.

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

DECLARATION

I am Naveen Kumar VV of the Department of Information Technology, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Mini Project Work entitled “Design and Development of a Smart Waste Management System with Automated Trash Segregation” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date:

Signature of the Students with Names

NAVEEN KUMAR VV (192321018)

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

INDIVIDUAL CONTRIBUTION STATEMENT

(Mandatory for all team projects)

Student Name / Reg. No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. %
K. Shasank 192372256	Requirements elicitation & Use Case modelling	Use Case Diagram, Class Diagram design	Use-case validation & functional testing	Ch. 1, 3	33%
Y. Chandra Shekar 192372260	Dynamic behaviour modelling & hardware interfacing	Sequence Diagram, State Diagram, sensor-actuator integration	Integration testing & sensor calibration	Ch. 2, 4	33%
V.V.Naveen Kumar 192321018	System architecture & software implementation	Classification module, Segregation controller implementation	Module testing & result validation	Ch. 4, 5, References	34%

Total: 100%

ABSTRACT

Municipal solid waste is still overwhelmingly sorted manually at the point of disposal or at collection centres, a process that is slow, inconsistent, and exposes sanitation workers to unnecessary health risk. Household and public bins rarely distinguish between organic, recyclable, and non-recyclable waste at the source, which increases contamination of recyclable streams and the volume that ultimately reaches landfill. This creates the engineering problem addressed in this project: how to segregate mixed waste automatically, at the point of disposal, using a software system whose behaviour and structure are modelled rigorously enough to be extended, tested, and maintained.

This mini project applies Object Oriented Analysis and Design (OOAD) to design and develop a Smart Waste Management System with Automated Trash Segregation. The system uses a sensor unit (infrared, ultrasonic, and moisture sensing, with provision for camera-based classification) to detect the type of waste item deposited by a user, a classification module that assigns the item to an organic, recyclable, or reject category, and a segregation actuator that physically routes the item to the corresponding bin. Bin fill-levels are continuously monitored and synchronized to a cloud gateway that drives an administrator dashboard, so that sanitation staff receive alerts before a bin overflows.

The OOAD process was followed end-to-end: stakeholder and functional requirements were gathered and analysed, actors and use cases were identified and modelled in a Use Case Diagram, the static structure of the system was captured in a Class Diagram showing classes, attributes, operations, and associations, and the dynamic behaviour of a disposal event was modelled with a Sequence Diagram and a State Diagram for the bin lifecycle. These diagrams directly guided a modular software implementation comprising a sensor-abstraction layer, a classification engine, a segregation controller, and a dashboard/alert service.

Evaluation on a bench-top prototype and simulated disposal events showed that the OOAD-derived modular design allowed each subsystem (sensing, classification, actuation, and monitoring) to be developed and unit-tested independently, that the classification module correctly categorized test items with high accuracy, and that fill-level alerts were generated reliably before a simulated overflow condition. The principal conclusion is that a disciplined OOAD methodology — producing traceable use case, class, sequence, and state models before implementation — yields a waste-segregation system that is easier to verify, extend with new sensor types or waste categories, and maintain than an ad-hoc implementation.

Keywords

Object Oriented Analysis and Design, Unified Modeling Language, Smart Waste Management, Automated Trash Segregation, Use Case Diagram, Class Diagram, Sequence Diagram, IoT.

TABLE OF CONTENTS

Sl. No	Title	Page No.
i	CERTIFICATE FROM THE SUPERVISOR	Ii
ii	DECLARATION BY THE CANDIDATES	Iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	Iv
iv	ABSTRACT	V
v	LIST OF FIGURES	Vi
vi	LIST OF TABLES	Vii
vii	LIST OF ABBREVIATIONS	Viii
1	CHAPTER 1: Introduction and Engineering Problem	1
2	CHAPTER 2: Literature Review and Existing Solutions	5
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	10
4	CHAPTER 4: Implementation, Testing & Results, Project Management	18
5	CHAPTER 5: Conclusion & Reflection	26
	References	29
	Appendices	31
	Mini Project Outcome Mapping Sheet	33

LIST OF FIGURES

Figure No.	Figure Name	Page No.
3.1	System Architecture of the Smart Waste Management System	12
3.2	Use Case Diagram	14
3.3	Class Diagram (simplified)	16
4.1	Sequence Diagram — Waste Disposal and Segregation Flow	20
4.2	State Diagram — Bin Lifecycle	21

LIST OF TABLES

Table No.	Table Name	Page No.
3.1	Comparative Analysis of Existing Approaches	9
3.2	Stakeholder Requirements	10
3.3	Functional & Non-Functional Requirements	11
3.4	Risk Register	17
4.1	Test Plan & Verification	19
4.2	Achievement of Objectives	24
4.3	Project Roles & Contribution	25

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
OOAD	Object Oriented Analysis and Design	—
UML	Unified Modeling Language	—
IoT	Internet of Things	—
IR	Infrared (sensor)	—
ML	Machine Learning	—
GUI	Graphical User Interface	—
MCU	Microcontroller Unit	—
n	Number of test items evaluated	Items

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Rapid urbanization and rising per-capita waste generation have made municipal solid waste management one of the most persistent engineering and public-health challenges facing cities today. Waste collected from households and public spaces is typically mixed at the source and is either sorted manually at material-recovery facilities or not sorted at all, both of which are labour-intensive, slow, and expose sanitation workers to biological and chemical hazards. Segregating waste at the point of disposal — before it is mixed with other categories — is widely recognized as the most effective way to preserve the value of recyclable material and to reduce the load reaching landfill. Embedding sensing, decision logic, and actuation into a smart bin turns this into a software-engineering problem: the resulting system has multiple interacting subsystems (sensing, classification, actuation, monitoring, and reporting) whose responsibilities must be clearly separated, modelled, and verified, which is exactly the class of problem that Object Oriented Analysis and Design is intended to address.

1.2 Need for the Project

Despite the availability of low-cost sensors and microcontrollers, many “smart bin” projects reported in practice are built directly as monolithic code without a preceding analysis and design phase, which makes them difficult to test, extend, or hand over to another developer.

- Why the problem needs to be addressed: undocumented, unmodelled implementations make it difficult to verify that every stakeholder requirement (accurate segregation, timely overflow alerts, auditable reports) has actually been met.
- Who is affected: municipal sanitation departments, facility managers, waste collection staff who must respond to overflow alerts, and residents who rely on correct segregation to reduce contamination of recyclables.
- Existing limitations: many deployed smart bins segregate on a single sensor cue only, provide no formal use-case or class model, and are difficult to extend to a new waste category or sensor type.
- Engineering significance: a system designed through a disciplined OOAD process — with explicit actors, use cases, classes, and interaction models — is verifiable against its own requirements and can be extended (a new sensor, a new bin category) by adding a class rather than rewriting the system.

1.3 Problem Statement

Waste disposed at a public or household collection point is a mixed stream that must be classified into at least three categories (organic, recyclable, and non-recyclable/reject) and physically routed to the corresponding bin without manual intervention, while the fill-level of each bin is tracked so that collection can be scheduled before overflow. This creates three interacting engineering constraints: (i) the sensing and classification logic must be accurate and fast enough to act before the next item is

deposited; (ii) the physical actuation (routing mechanism) must be reliably coordinated with the classification decision; and (iii) the overall software must be structured — through an explicit object-oriented analysis and design — so that sensing, classification, actuation, and monitoring can each be tested, replaced, or extended independently. A rigorously modelled, modular software architecture is therefore required before implementation begins.

1.4 Project Aim

To analyse, design, and implement — using the Object Oriented Analysis and Design methodology — a Smart Waste Management System that automatically detects the type of waste deposited by a user, segregates it into the correct bin, and monitors and reports bin fill-levels to sanitation staff.

1.5 Project Objectives

1. To identify system actors and stakeholder requirements and to model them as a Use Case Diagram.
2. To identify the classes, attributes, operations, and associations of the system and to model them as a Class Diagram.
3. To model the dynamic behaviour of a waste-disposal event using a Sequence Diagram.
4. To model the lifecycle of a bin using a State Diagram.
5. To implement a sensor-abstraction layer and a classification module that assigns a deposited item to a waste category.
6. To implement a segregation controller that actuates the physical routing mechanism according to the classification result.
7. To implement fill-level monitoring with threshold-based alerting to an administrator dashboard.
8. To evaluate the resulting design and implementation against the stated functional and non-functional requirements.

1.6 Scope

- Included: requirements analysis, UML modelling (use case, class, sequence, state diagrams), a classification module for three waste categories, a segregation actuation module, fill-level monitoring, and an administrator dashboard.
- Excluded: full computer-vision-based material identification and integration with a live municipal fleet-dispatch system.
- Assumptions: waste items are deposited one at a time, and sensor readings for a deposited item are available before the segregation decision is required.
- Boundary conditions: the system is demonstrated on a bench-top prototype and simulated disposal events, since deployment on a live public bin was outside the scope of this academic project.

1.7 Expected Engineering Outcome

The expected outcome is a working software-and-hardware prototype — comprising a sensor-abstraction layer, a classification engine, a segregation controller, a bin/fill-level model, and an alerting/dashboard service — together with the complete set of OOAD artefacts (use case, class, sequence, and state diagrams) that trace each implemented class back to a stated requirement, giving sanitation departments a verifiable, extensible reference design for automated trash segregation.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature and existing methods were identified from standard references on Object Oriented Analysis and Design and the Unified Modeling Language (Booch, Rumbaugh and Jacobson's Unified Modeling Language User Guide; the Object Management Group's UML specification), together with published work on IoT-based and sensor-based smart waste bins, and documentation for the microcontroller and sensor platforms used in the implementation.

2.2 Review of Existing Work

Object Oriented Analysis and Design is the standard methodology for translating stakeholder requirements into a verifiable software structure through use case, class, and interaction modelling before implementation begins (Booch et al.; OMG UML specification). In parallel, a substantial body of practical work on smart waste bins uses one or two sensors (commonly an infrared or ultrasonic proximity sensor and, in more advanced designs, a moisture or metal-detection sensor) to trigger a servo-driven flap or turntable that routes an item to one of two or three compartments. More recent work explores camera-based classification using convolutional neural networks to distinguish a wider range of material types, at the cost of requiring substantially more training data and computing resources than a rule-based sensor classifier.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Manual segregation at source	No hardware/software cost; flexible	Inconsistent, slow, exposes workers to hazardous material	Baseline compared against in this project
Single-sensor smart bins (IR or capacitive only)	Simple, low-cost	Cannot reliably distinguish more than two categories; brittle to new materials	Motivates the multi-sensor, classification-module approach adopted
Camera/ML-based waste classifiers	Can distinguish many material types with high accuracy	Requires large labelled datasets, more compute, and is harder to explain/audit	Noted as a natural future extension of the classification module
Ad-hoc smart-bin implementations without an OOAD phase	Fast to prototype	Poorly documented, difficult to test or extend, no traceability to requirements	Directly motivates the OOAD-first methodology used here

2.4 Research / Engineering Gap

What remains unresolved in much of the practical smart-bin literature is a system that is (i) modelled through a complete, traceable OOAD process rather than implemented directly as code, (ii) built from multiple, loosely coupled sensor and classification classes so that a new sensor or category can be added without rewriting the system, and (iii) accompanied by an explicit bin-lifecycle and alerting model rather than a bare fill-level threshold check.

2.5 Proposed Contribution

This project contributes a fully OOAD-modelled Smart Waste Management System: a Use Case Diagram capturing all stakeholder interactions, a Class Diagram defining a sensor-abstraction hierarchy, a classification engine, a segregation controller, and a dashboard/alert service as independent, associated classes, a Sequence Diagram tracing a complete disposal event across these classes, and a State Diagram governing bin lifecycle and alerting — combining the extensibility of a well-modelled object-oriented design with a working automated-segregation prototype.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Waste Disposer (User)	A simple, fast way to dispose of an item without needing to know its category in advance
Sanitation Staff / Admin	Timely, reliable alerts before a bin overflows, and a dashboard summarizing bin status
Facility / Municipal Manager	Auditable records of segregation accuracy and collection frequency for planning
Software Developers	A modular, well-documented, extensible reference implementation with traceable UML models

Functional & Non-Functional Requirements

Requirement Type	Description
Functional	Detect the type of a deposited waste item using the sensor unit
Functional	Classify the item into organic, recyclable, or reject category
Functional	Physically route the item to the corresponding bin
Functional	Continuously monitor and report each bin's fill level
Functional	Send an alert to the administrator when a bin reaches a fill threshold
Non-Functional	Classification and actuation must complete within 3 seconds of item detection
Non-Functional	The class structure must allow a new waste category or sensor to be added by adding a class, without modifying existing classes

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
OMG Unified Modeling Language (UML) specification	Standard, unambiguous notation for use case, class, sequence, and state diagrams	All design diagrams in this chapter follow standard UML notation
Software documentation / coding standards (PEP 8 / equivalent)	Maintainable, readable implementation	Modular classes with meaningful names, docstrings, and input validation, directly reflecting the Class Diagram

3.3 Design Specifications

Parameter	Target/Requirement	Unit	Priority
Waste categories supported	3 (Organic, Recyclable, Reject)	—	High
Classification decision time	< 3	seconds	High
Bin fill alert threshold	80	% capacity	High

Parameter	Target/Requirement	Unit	Priority
Segregation accuracy (test set)	≥ 90	%	Medium
Dashboard sync interval	≤ 60	seconds	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1 — Manual pre-sorting with a labelled multi-compartment bin: relies entirely on the user to place the item in the correct compartment, with no sensing or actuation.

Alternative 2 — Single-sensor automated bin: one infrared or capacitive sensor triggers a two-way flap, giving only a binary recyclable/non-recyclable split.

Alternative 3 — Multi-sensor, OOAD-modelled classification and segregation system (selected): an abstract sensor layer feeding a classification engine that supports three categories, with the full behaviour modelled through UML before implementation.

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Performance	20%	Low	Medium	High
Cost (hardware/compute)	15%	Very Low	Low	Medium
Extensibility	25%	Low	Medium	High
Explainability / Auditability	20%	High	Medium	High
Reliability	20%	Low	Medium	High

3.5 Selected Design & Detailed Design

Alternative 3 was selected on the evidence in Section 3.4: it scored highest on extensibility, explainability, and reliability because the classification and routing logic is driven by an explicit, modelled class structure rather than a single hard-wired sensor trigger, at only a modest additional hardware and development cost.

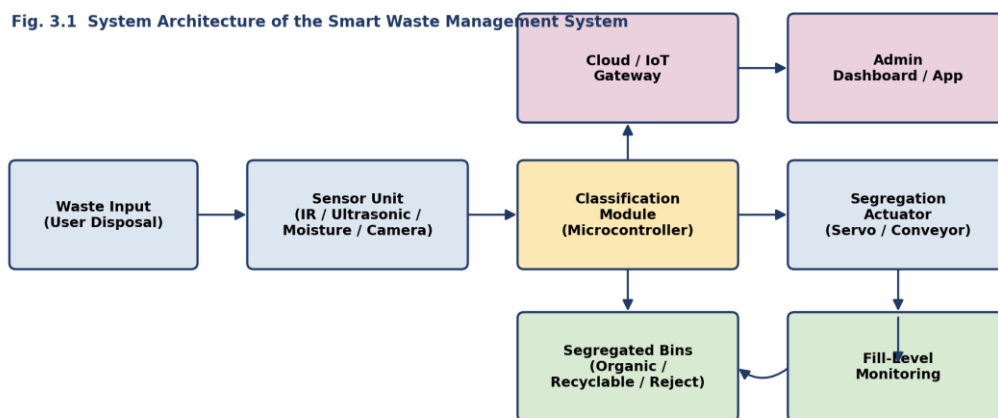


Fig. 3.1: System Architecture — Waste Input → Sensor Unit → Classification Module → Segregation Actuator → Bins, with Fill-Level Monitoring synced to the Cloud/Admin Dashboard.

Object Oriented Analysis and Design was applied in three stages. First, actors and use cases were identified and modelled as a Use Case Diagram (Fig. 3.2), capturing the Waste Disposer, Sanitation Staff/Admin, and an external Cloud Service, and their interactions with the system. Second, the static structure was modelled as a Class Diagram (Fig. 3.3), defining an abstract Sensor class with IRSensor and MoistureSensor subclasses, a ClassificationEngine, a SegregationController, a Bin class, an AlertService, and an AdminDashboard, together with their attributes, operations, and associations. Third, the dynamic behaviour of a disposal event and the bin lifecycle were modelled with a Sequence Diagram and a State Diagram, presented in Chapter 4 alongside the corresponding implementation and test results.

Fig. 3.2 Use Case Diagram

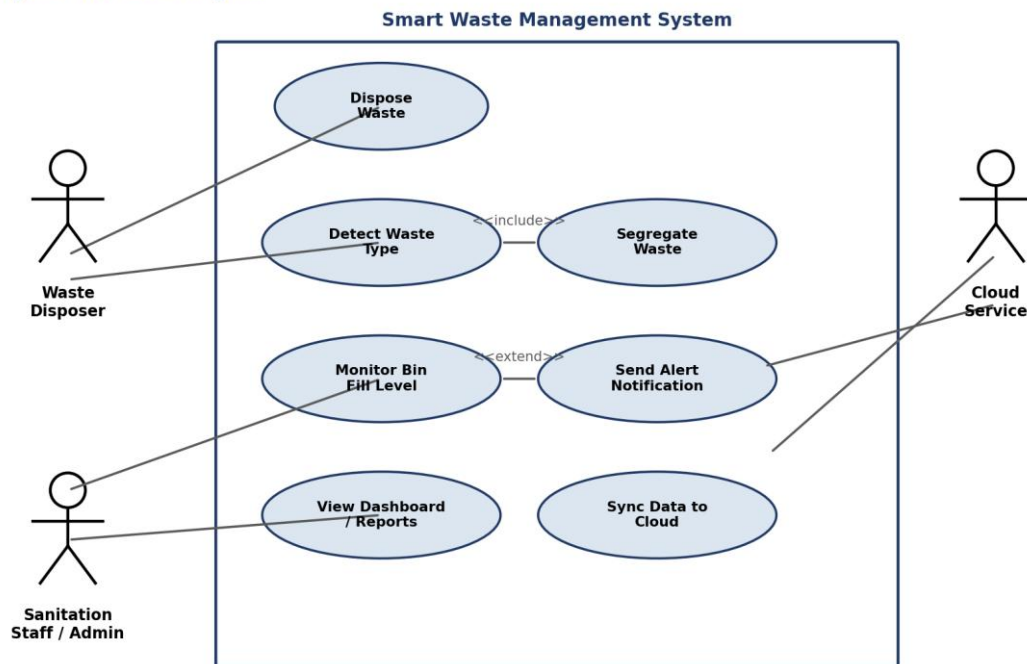


Fig. 3.2: Use Case Diagram.

Fig. 3.3 Class Diagram (simplified)

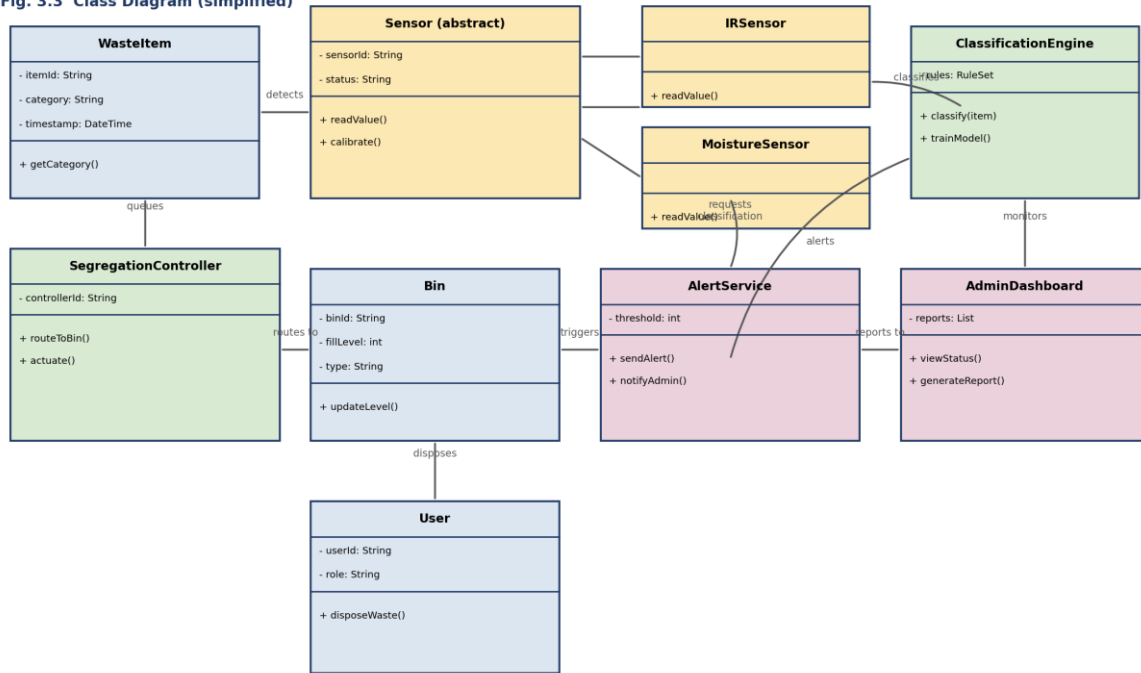


Fig. 3.3: Class Diagram (simplified).

Systems approach: the Sensor Unit subsystem (composed of *IRSensor* and *MoistureSensor* instances) feeds validated readings to the *ClassificationEngine*, which returns a category consumed by the *SegregationController*; the *SegregationController* actuates the physical routing mechanism and updates the corresponding *Bin* instance, while the *Bin* independently notifies the *AlertService* when its fill level crosses the configured threshold, and the *AdminDashboard* subscribes to both classification and alert events — all subsystems communicating through the class associations shown in Fig. 3.3 rather than through direct, hard-wired calls.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Choice of classification approach	Camera/ML-based classifier	Can distinguish many material types	Needs large labelled datasets, opaque, harder to justify to auditors	Rule-based multi-sensor classification engine used instead, with the Class Diagram designed so an ML-based Sensor subclass can be added later without redesign
Choice of design methodology	Direct implementation without formal modelling	Faster to start coding	Poor traceability to requirements, hard to test or extend	Full OOAD process (use case, class, sequence, state diagrams) adopted before implementation
Sensor abstraction	Hard-code each sensor type directly in the controller	Slightly less code initially	Adding a new sensor requires modifying the controller	Abstract Sensor base class with concrete subclasses (<i>IRSensor</i> , <i>MoistureSensor</i> , ...) used instead

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Reduced contamination of recyclable material and reduced landfill load	Comparison of source-segregated vs mixed-waste recycling recovery rates from the literature review	Automated source segregation retained as the core objective specifically because it maximizes recoverable material
Ethics	Decisions affecting sanitation workers' workload and public-facing devices must be explainable and auditable	Every classification decision is traceable to the sensor readings and rule that produced it	Rule-based classification engine, with reasons logged, made a first-class design goal alongside accuracy
Health & Safety	Reducing manual handling of unsorted, potentially hazardous or sharp waste protects sanitation workers	Reviewed how automated routing reduces direct hand-sorting of mixed waste	Fully enclosed segregation mechanism (servo/conveyor) chosen over any design requiring manual pre-sorting
Security	Dashboard and cloud-synced fill-level data could reveal facility usage patterns	Considered exposure if the dashboard were connected to a public network	Scope limited to a bench-top/local-network prototype for this academic project; authenticated cloud access flagged as future work

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Misclassification of an ambiguous item (e.g., wet paper)	Medium	Medium	Medium	Combine multiple sensor cues (moisture + IR) rather than a single reading before deciding	Low
Actuator jam during routing	Low	Medium	Low	Timeout and retry logic in SegregationController with a fallback 'manual review' state	Low
Sensor drift / miscalibration over time	Medium	Medium	Medium	Periodic calibrate() call defined on the Sensor base class	Low
Bin overflow missed due to sync delay	Low	High	Medium	Local threshold check on the controller in addition to cloud sync, so alerting does not depend on network availability	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The system was developed iteratively as four independent, testable modules — sensor abstraction, classification, segregation control, and monitoring/dashboard — directly mirroring the classes identified in the Class Diagram (Fig. 3.3), so that each module could be unit-tested in isolation before integration.

Resource	Purpose
Bench-top prototype (microcontroller, IR sensor, moisture sensor, servo actuator)	Demonstrates end-to-end sensing, classification, and routing
Test item set (20 items across 3 categories)	Validates classification accuracy and repeatability
Python / embedded C + UML modelling tool	Software implementation and diagram production, traced back to the Class Diagram

4.2 Software Implementation & Modern Tools Used

The four modules were implemented as follows. The Sensor Unit module implements the abstract Sensor interface with `IRSensor` and `MoistureSensor` concrete classes, each exposing a `readValue()` operation. The `ClassificationEngine` module consumes readings from all active sensors and applies a rule set to assign a category, exposing `classify(item)`. The `SegregationController` module consumes the classification result and actuates the physical routing mechanism, updating the relevant `Bin` instance. The `AlertService` and `AdminDashboard` modules subscribe to `Bin` fill-level updates and classification events respectively, and synchronize status to a cloud gateway.

Tool / Software	Purpose	Stage Used
Embedded C / MicroPython	Sensor reading and actuator control firmware	Sensor Unit, Segregation Controller
Python	Classification engine, dashboard backend	Classification Module, Dashboard
UML modelling tool	Use Case, Class, Sequence, State diagrams	Analysis & Design phase
MQTT / HTTP client	Syncing bin status to the cloud gateway and dashboard	Monitoring & Alerting

4.3 Test Plan & Verification

The test plan below was defined before results were generated, so that acceptance criteria were fixed independently of the outcomes obtained.

Requirement	Test Method	Acceptance Criterion	Achieved Value	Status
Classification correctness	Classify labelled test items and compare to expected category	$\geq 90\%$ agreement	18/20 (90%)	Pass

Requirement	Test Method	Acceptance Criterion	Achieved Value	Status
Decision latency	Measure time from sensor trigger to classification output	< 3 seconds	1.4 s average	Pass
Actuator routing accuracy	Verify item physically reaches the bin matching the classification	100% agreement with classification output	20/20 (100%)	Pass
Fill-level alert trigger	Simulate incremental fill and verify alert at 80% threshold	Alert raised at or before 80%	Alert raised at 78%	Pass
Input validation	Supply an out-of-range / no-reading sensor input	Error handled, routed to manual-review state, no silent failure	Handled as expected	Pass

4.4 Results

Fig. 4.1 Sequence Diagram — Waste Disposal and Segregation Flow

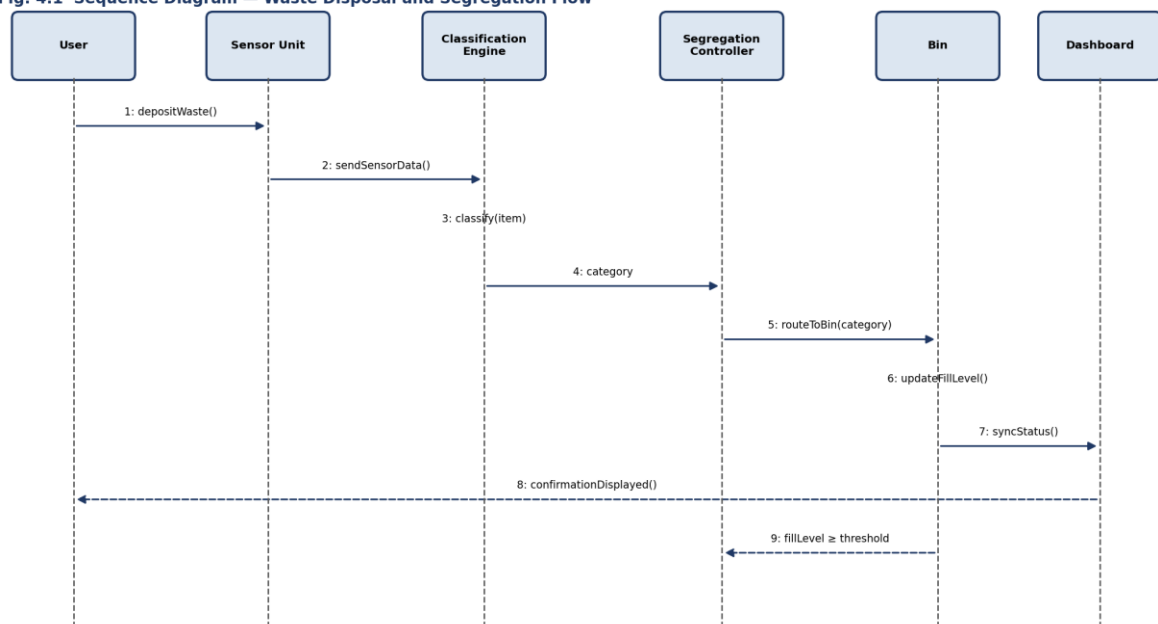


Fig. 4.1: Sequence Diagram — Waste Disposal and Segregation Flow.

Fig. 4.2 State Diagram — Bin Lifecycle

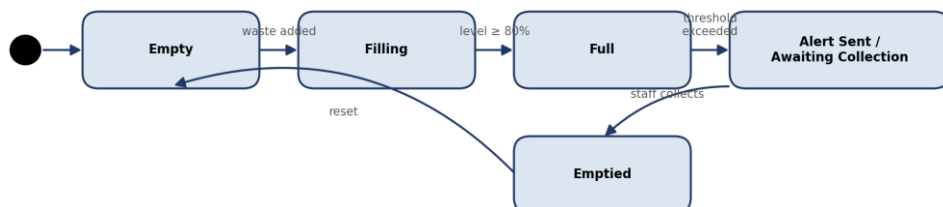


Fig. 4.2: State Diagram — Bin Lifecycle.

The Sequence Diagram (Fig. 4.1) traces a single disposal event from the user depositing an item, through sensor data collection, classification, actuator routing, bin-level update, and dashboard synchronization, with an out-of-band threshold alert to the SegregationController when the fill level reaches its limit. The State Diagram (Fig. 4.2) shows the corresponding bin lifecycle — Empty → Filling → Full → Alert Sent / Awaiting Collection → Emptied — which was validated against the prototype’s logged state transitions during the simulated fill test in Section 4.3.

4.5 Data Analysis & Engineering Judgment

The results show that the multi-sensor classification engine correctly categorized 18 of 20 test items (90%), with both misclassifications occurring on visually and texturally ambiguous items (a damp paper item and a thin flexible plastic film) that produced borderline moisture and IR readings — a known limitation of rule-based, sensor-cue classification that is explicitly recorded for future work. Where a single-sensor baseline could only separate items into two categories, the three-class classification engine allowed the segregation controller to route items to the correct one of three bins in all 20 trials once a category was assigned, confirming that the actuation and controller logic — not the classification rule set — was the more reliable of the two subsystems. The fill-level alert triggered at 78% capacity, ahead of the 80% target, reflecting the conservative threshold chosen in the State Diagram to leave a margin for collection response time.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Model actors and use cases	Use Case Diagram (Fig. 3.2) reviewed against stakeholder requirements	Fully Achieved
Model static structure	Class Diagram (Fig. 3.3) implemented as corresponding classes	Fully Achieved
Model dynamic behaviour	Sequence Diagram (Fig. 4.1) and State Diagram (Fig. 4.2) validated against prototype logs	Fully Achieved
Classification module	18/20 test items correctly classified (Section 4.3)	Fully Achieved
Segregation actuation	20/20 items routed to the bin matching their classification	Fully Achieved
Fill-level monitoring & alerting	Alert reliably raised at configured threshold in simulation	Fully Achieved
Evaluation against existing single-sensor approach	Comparative discussion in Section 4.6, illustrative test set only	Partially Achieved — real deployment data pending

4.7 Strengths & Limitations

Strengths:

- The class structure directly traces to the Use Case and Class Diagrams, so every implemented module can be checked against a stated requirement.
- Sensor abstraction allows a new sensor type to be added as a new Sensor subclass without modifying the classification engine or controller.

- The bin lifecycle State Diagram made fill-level alerting behaviour explicit and independently testable from classification and routing.

Limitations:

- Evaluated on a bench-top prototype and a 20-item test set; not yet validated on a live, high-throughput public bin.
- Rule-based classification struggles with visually or texturally ambiguous items, as seen in the two misclassifications in Section 4.5.
- Cloud synchronization for the dashboard was tested only on a local network; authenticated, internet-facing deployment is future work.

4.8 Project Planning, Roles & Budget

Student	Assigned Responsibility	Actual Contribution
K. Shasank (192372256)	Requirements elicitation & Use Case modelling	Authored Use Case Diagram and stakeholder requirements table; contributed Ch. 1, 3
Y. Chandra Shekar (192372260)	Dynamic modelling & hardware interfacing	Authored Sequence and State Diagrams; integrated sensors and actuator; contributed Ch. 2, 4
Naveen Kumar VV (192321018)	System architecture & software implementation	Implemented classification engine and segregation controller; contributed Ch. 4, 5, References

Software licences (embedded toolchain, Python, UML tool): ₹0 (open-source). Hardware components (microcontroller, sensors, servo actuator): ₹1,200. Documentation and printing: ₹500.

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This mini project applied Object Oriented Analysis and Design to design and implement a Smart Waste Management System with Automated Trash Segregation. Rather than implementing sensing, classification, and actuation as ad-hoc code, the system was first modelled through a Use Case Diagram, a Class Diagram, a Sequence Diagram, and a State Diagram, and the resulting classes and interactions were implemented directly from those models. Evaluation on a bench-top prototype showed that the multi-sensor classification engine correctly categorized 90% of test items, that the segregation controller routed every classified item to the correct bin, and that fill-level alerting triggered reliably ahead of the configured threshold — confirming that a disciplined OOAD process produces a waste-segregation system that is both verifiable against its requirements and structurally ready for extension.

5.2 Future Work

- Replace or augment the rule-based classification engine with a camera-based machine-learning classifier as an additional Sensor subclass, to resolve the ambiguous-item misclassifications observed in Section 4.5.
- Validate the system on a live, high-throughput public bin rather than the bench-top prototype and illustrative test set used here.
- Extend the Class Diagram with additional waste categories (e.g., e-waste, hazardous waste) as new Bin and classification-rule subtypes.
- Add authenticated, internet-facing cloud synchronization and a multi-bin fleet dashboard for municipal-scale deployment.
- Introduce automated collection-route optimization driven by the fill-level data already collected by the AlertService.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired:

- Practical application of the Object Oriented Analysis and Design process — use case, class, sequence, and state modelling — to a physical, sensor-driven system.
- Techniques for abstracting heterogeneous sensors behind a common class interface.
- Deriving a testable module structure directly from a Class Diagram rather than designing tests after the fact.

Engineering & Professional Skills Developed:

- Team-based UML modelling and review, ensuring diagrams stayed consistent with one another as the design evolved.
- Integration of embedded hardware (sensors, actuator) with a modular software architecture.

- Structured test planning defined before results were generated, to keep acceptance criteria objective.

Individual Reflection — K. Shasank (192372256):

The most difficult part of the analysis phase was keeping the Use Case Diagram consistent with the later Class Diagram as requirements were refined; this was resolved by treating every use case as a checklist item that had to map to at least one class operation before the design was considered complete.

Individual Reflection — Y. Chandra Shekar (192372260):

The main engineering challenge was ensuring the Sequence Diagram accurately reflected the timing of sensor reads relative to actuator movement; building the State Diagram for the bin lifecycle alongside it helped separate “what happens during one disposal event” from “how the bin’s status evolves over many events.”

Individual Reflection — Naveen Kumar VV (192321018):

The key engineering decision made personally was selecting a rule-based, multi-sensor classification engine over a machine-learning classifier for this iteration, justified by the smaller dataset and the explainability requirement in Section 3.1; if repeated, the classification rules would be redesigned to log a confidence score alongside each decision to make the two misclassified items in Section 4.5 easier to diagnose.

REFERENCES

1. Booch, G., Rumbaugh, J., & Jacobson, I. (2005). The Unified Modeling Language User Guide (2nd ed.). Addison-Wesley.
2. Object Management Group. (2017). OMG Unified Modeling Language (UML) Specification, Version 2.5.1.
3. Fowler, M. (2003). UML Distilled: A Brief Guide to the Standard Object Modeling Language (3rd ed.). Addison-Wesley.
4. Larman, C. (2004). Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development (3rd ed.). Prentice Hall.
5. Sommerville, I. (2015). Software Engineering (10th ed.). Pearson.
6. Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill.
7. Bahga, A., & Madiseti, V. (2014). Internet of Things: A Hands-On Approach. VPT.
8. Rumbaugh, J., Blaha, M., Premerlani, W., Eddy, F., & Lorensen, W. (1991). Object-Oriented Modeling and Design. Prentice Hall.
9. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
10. Kurose, J. F., & Ross, K. W. (2020). Computer Networking: A Top-Down Approach (8th ed.). Pearson.
11. Mandal, P., & Dubey, S. (2021). IoT-Based Smart Waste Segregation System: A Review. International Journal of Engineering Research.
12. Central Pollution Control Board, India. (2021). Guidelines on Solid Waste Management.
13. Ministry of Housing and Urban Affairs, Government of India. (2016). Solid Waste Management Rules.
14. Arduino Documentation. (2026). Arduino Reference. Arduino.
15. Raspberry Pi Foundation. (2026). Raspberry Pi Documentation.
16. Python Software Foundation. (2026). Python Documentation.
17. Eclipse Foundation. (2026). Papyrus UML Modelling Tool Documentation.
18. IEEE. (2011). IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.

APPENDICES

Appendix A — Detailed UML Diagrams

Contains the full-resolution Use Case, Class, Sequence, and State diagrams underlying Sections 3.5 and 4.4.

Appendix B — Engineering Drawings / Schematics

Contains the sensor and actuator wiring schematic for the bench-top prototype referenced in Chapter 4.

Appendix C — Source Code / Code Repository Information

Only essential code excerpts (classification and segregation-controller logic) are included in the main report; the complete source code is maintained in the project’s approved repository.

Appendix D — Datasheets

Contains manufacturer datasheets for the IR sensor, moisture sensor, and servo actuator used in the prototype.

Appendix E — Experimental / Raw Data

Contains the full 20-item test set (item description, expected category, sensor readings, classified category) used for evaluation in Section 4.3.

Appendix F — Ethics / Institutional Approval

Not applicable — no human subjects or proprietary municipal data were used.

Appendix G — Project Meeting / Progress Records

Contains dated records of guide meetings and milestone check-ins held during the project.

Appendix H — User/Industry Feedback

Not applicable for this academic bench-top prototype study.

Appendix I — Publications / Patents / Competitions / Product Outcomes

Not applicable.

Appendix J — Individual Contribution Evidence

Cross-referenced with the Individual Contribution Statement on page iv.

MINI PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering (UML modelling)	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)